

[Power BI M Language (Power Query)] (CheatSheet)

1. Data Source Connections

- Connect to CSV: `Csv.Document(File.Contents("C:\data.csv"), [Delimiter=",", Columns=5, Encoding=1252, QuoteStyle=QuoteStyle.Csv])`
- Connect to Excel: `Excel.Workbook(File.Contents("C:\data.xlsx"), null, true)`
- Connect to SQL Server: `Sql.Database("server", "database", [Query="SELECT * FROM table"])`
- Connect to Web API: `Json.Document(Web.Contents("https://api.example.com/data"))`
- Connect to SharePoint: `SharePoint.Tables("https://company.sharepoint.com/sites/site", [ApiVersion = 15])`

2. Table Operations

- Remove columns: `Table.RemoveColumns(Source, {"Column1", "Column2"})`
- Keep columns: `Table.SelectColumns(Source, {"Column1", "Column2"})`
- Rename column: `Table.RenameColumns(Source, {"OldName", "NewName"})`
- Reorder columns: `Table.ReorderColumns(Source, {"Column3", "Column1", "Column2"})`
- Add column: `Table.AddColumn(Source, "NewColumn", each [Column1] * 2)`
- Remove top rows: `Table.Skip(Source, 5)`
- Keep top rows: `Table.FirstN(Source, 10)`
- Remove bottom rows: `Table.RemoveLastN(Source, 5)`
- Keep bottom rows: `Table.LastN(Source, 10)`
- Remove duplicate rows: `Table.Distinct(Source)`

3. Data Cleaning

- Replace values: `Table.ReplaceValue(Source, "Old", "New", Replacer.ReplaceText, {"Column1", "Column2"})`
- Remove errors: `Table.RemoveRowsWithErrors(Source)`
- Fill down values: `Table.FillDown(Source, {"Column1", "Column2"})`
- Fill up values: `Table.FillUp(Source, {"Column1", "Column2"})`
- Handle null values: `Table.ReplaceValue(Source, null, "N/A", Replacer.ReplaceValue, {"Column1"})`
- Trim whitespace: `Table.TransformColumns(Source, {"Column1", Text.Trim})`
- Change case: `Table.TransformColumns(Source, {"Column1", Text.Upper})`
- Remove special characters: `Table.TransformColumns(Source, {"Column1", each Text.Select(_, {"A".."Z", "a".."z", "0".."9"})})`

4. Data Transformation

- **Split column by delimiter:** `Table.SplitColumn(Source, "Column1", Splitter.SplitTextByDelimiter(",", { "Column1.1", "Column1.2" })`
- **Merge columns:** `Table.AddColumn(Source, "MergedColumn", each [Column1] & " " & [Column2])`
- **Pivot column:** `Table.Pivot(Source, List.Distinct(Source[Column1]), "Column1", "Value", List.Sum)`
- **Unpivot columns:** `Table.UnpivotOtherColumns(Source, {"ID"}, "Attribute", "Value")`
- **Transpose table:** `Table.Transpose(Source)`
- **Group rows:** `Table.Group(Source, {"Column1"}, {"Count", each Table.RowCount(_), Int64.Type})`
- **Expand column:** `Table.ExpandRecordColumn(Source, "Column1", {"Field1", "Field2"})`

5. Date and Time Operations

- **Extract year:** `Table.AddColumn(Source, "Year", each Date.Year([Date]))`
- **Extract month:** `Table.AddColumn(Source, "Month", each Date.Month([Date]))`
- **Extract day:** `Table.AddColumn(Source, "Day", each Date.Day([Date]))`
- **Extract weekday:** `Table.AddColumn(Source, "Weekday", each Date.DayOfWeek([Date]))`
- **Format date:** `Table.AddColumn(Source, "FormattedDate", each Date.ToText([Date], "yyyy-MM-dd"))`
- **Calculate age:** `Table.AddColumn(Source, "Age", each Duration.Years(Duration.From(DateTime.LocalNow() - [BirthDate])))`
- **Add days:** `Table.AddColumn(Source, "NewDate", each Date.AddDays([Date], 7))`
- **Date difference:** `Table.AddColumn(Source, "DaysBetween", each Duration.Days([EndDate] - [StartDate]))`

6. Text Operations

- **Extract substring:** `Table.AddColumn(Source, "Substring", each Text.Start([Text], 5))`
- **Find text position:** `Table.AddColumn(Source, "Position", each Text.PositionOf([Text], "search"))`
- **Replace text:** `Table.AddColumn(Source, "ReplacedText", each Text.Replace([Text], "old", "new"))`
- **Split text:** `Table.AddColumn(Source, "SplitText", each Text.Split([Text], ","))`
- **Combine text:** `Table.AddColumn(Source, "CombinedText", each Text.Combine([TextList], ", "))`
- **Proper case:** `Table.TransformColumns(Source, {"Column1", Text.Proper})`
- **Remove diacritics:** `Table.AddColumn(Source, "NoDiacritics", each Text.RemoveDiacritics([Text]))`

7. Number Operations

- **Round numbers:** `Table.TransformColumns(Source, {"Column1", each Number.Round(_, 2)})`
- **Floor numbers:** `Table.TransformColumns(Source, {"Column1", each Number.RoundDown(_)})`
- **Ceiling numbers:** `Table.TransformColumns(Source, {"Column1", each Number.RoundUp(_)})`
- **Absolute value:** `Table.TransformColumns(Source, {"Column1", each Number.Abs(_)})`
- **Percentage:** `Table.AddColumn(Source, "Percentage", each [Value] / 100)`

8. Conditional Operations

- **If-then-else:** `Table.AddColumn(Source, "Category", each if [Value] > 100 then "High" else "Low")`
- **Multiple conditions:** `Table.AddColumn(Source, "Category", each if [Value] > 100 then "High" else if [Value] > 50 then "Medium" else "Low")`
- **Switch statement:** `Table.AddColumn(Source, "Category", each Table.FromRows({[Value]}), {"Value"}) {[Value]})`

9. List Operations

- **Create list:** `{1, 2, 3, 4, 5}`
- **List.Generate:** `List.Generate(() => 1, each _ <= 10, each _ + 1)`
- **List from range:** `{1..10}`
- **List.Transform:** `List.Transform({1, 2, 3}, each _ * 2)`
- **List.Select:** `List.Select({1, 2, 3, 4, 5}, each _ % 2 = 0)`
- **List.Sum:** `List.Sum({1, 2, 3, 4, 5})`
- **List.Average:** `List.Average({1, 2, 3, 4, 5})`

10. Error Handling

- **Try-otherwise:** `try [Column1] otherwise null`
- **Replace errors:** `Table.ReplaceErrorValues(Source, {"Column1", "Error"})`
- **Remove rows with errors:** `Table.RemoveRowsWithErrors(Source)`
- **Catch errors:** `Table.AddColumn(Source, "HasError", each try [Column1] otherwise "Error")`

11. Custom Functions

- **Define function:** `(x) => x * 2`
- **Function with multiple parameters:** `(x, y) => x + y`
- **Recursive function:** `(n) => if n <= 1 then 1 else n * @(n-1)`

12. Advanced Table Operations

- **Join tables:** `Table.Join(Table1, "Key", Table2, "Key")`
- **Fuzzy join:** `Table.FuzzyJoin(Table1, "Name", Table2, "Name", [IgnoreCase=true, IgnoreSpace=true])`
- **Merge queries:** `Table.NestedJoin(Table1, "Key", Table2, "Key", "NewColumn", JoinKind.LeftOuter)`
- **Append queries:** `Table.Combine({Table1, Table2})`
- **Buffer table:** `Table.Buffer(Source)`
- **Table from list:** `Table.FromList({1, 2, 3}, Splitter.SplitByNothing(), {"Column1"})`
- **Table from records:** `Table.FromRecords([ID=1, Name="John"], [ID=2, Name="Jane"])]`

13. Type Casting and Conversion

- **Change type:** `Table.TransformColumnTypes(Source, {"Column1", Int64.Type})`
- **Parse JSON:** `Table.AddColumn(Source, "ParsedJSON", each Json.Document([JSONColumn]))`
- **Convert to table:** `Table.FromList(Source, Splitter.SplitByNothing(), null, null, ExtraValues.Error)`
- **Convert to record:** `Record.FromList(Source, {"Field1", "Field2", "Field3"})`
- **Convert to list:** `List.FromTable(Source, "Column1")`

14. Web Scraping and API Calls

- **Web scraping:** `Web.Page(Web.Contents("https://example.com"))[Data]`
- **API call with parameters:** `Json.Document(Web.Contents("https://api.example.com", [Query=[param="value"]]))`
- **Paginated API call:** `List.Generate(() => [i=0, res = null], each [i] < 10, each [i=[i]+1, res = Json.Document(Web.Contents("https://api.example.com?page=" & Text.From([i])))], each [res])`

15. File Operations

- **List files in folder:** `Folder.Files("C:\Data")`
- **Combine files:** `Folder.Files("C:\Data") {[Content]}`
- **Get file metadata:** `Table.AddColumn(Source, "FileInfo", each File.Info([FilePath]))`

16. Database Operations

- **Execute SQL:** `Value.NativeQuery(SqlServer.Database("server", "database"), "SELECT * FROM table")`
- **Stored procedure:** `SqlServer.Database("server", "database"){[Schema="dbo",Item="StoredProcedure"]}(Parameter1=1, Parameter2="value")`

17. Customizing Data Preview

- **Set data preview:** `Table.FirstN(Source, 100)`
- **Disable data preview:** `#table({"Column1"}, {})`

18. Performance Optimization

- **Index column:** `Table.AddIndexColumn(Source, "Index", 0, 1)`
- **Keep columns:** `Table.SelectColumns(Source, {"Column1", "Column2"}, MissingField.Ignore)`
- **Remove other columns:** `Table.SelectColumns(Source, {"Column1", "Column2"})`
- **Change column data type:** `Table.TransformColumnTypes(Source, {"Column1", type text}, {"Column2", Int64.Type}))`

19. Data Profiling

- **Column profile:** `Table.Profile(Source)`
- **Column distribution:** `Table.Group(Source, {"Column1"}, {"Count", each Table.RowCount(_), type number}))`
- **Detect data type:** `Type.DetectFromValue([Value])`

20. Advanced Techniques

- **Dynamic column selection:** `Table.SelectColumns(Source, List.Select(Table.ColumnNames(Source), each Text.StartsWith(_, "prefix")))`
- **Custom aggregation:** `Table.Group(Source, {"Category"}, {"CustomAgg", each List.Max(List.RemoveNulls([Value])), type number}))`
- **Pivot with custom aggregation:** `Table.Pivot(Source, List.Distinct(Source[Category]), "Category", "Value", each List.Sum(List.RemoveNulls(_)))`
- **Dynamic unpivot:** `Table.UnpivotOtherColumns(Source, {"ID"}, "Attribute", "Value")`
- **Fuzzy matching:** `Table.AddColumn(Source, "FuzzyMatch", each List.First(List.Sort(Table.ToList(LookupTable), (x) => Text.FuzzyEquals([Name], x{1}))))`
- **Custom sorting:** `Table.Sort(Source, {"Column1", (x) => List.PositionOf({"Low", "Medium", "High"}, x)})`

21. Advanced Date and Time Manipulations

- **Fiscal year calculation:** `Table.AddColumn(Source, "FiscalYear", each if Date.Month([Date]) >= 7 then Date.Year([Date]) + 1 else Date.Year([Date]))`
- **ISO week number:** `Table.AddColumn(Source, "ISOWeek", each Date.WeekOfYear([Date], Day.Monday))`
- **Business days between dates:** `Table.AddColumn(Source, "BusinessDays", each Duration.Days(Duration.From([EndDate] - [StartDate])) - (Duration.Days(Duration.From([EndDate] - [StartDate])) / 7) * 2)`
- **Rolling date window:** `Table.SelectRows(Source, each [Date] >= Date.AddYears(Date.From(DateTime.LocalNow()), -1) and [Date] <= Date.From(DateTime.LocalNow()))`

22. Advanced Text Manipulations

- **Extract domain from email:** `Table.AddColumn(Source, "Domain", each Text.AfterDelimiter([Email], "@"))`
- **Concatenate with delimiter:** `Table.AddColumn(Source, "FullName", each Text.Combine({[FirstName], [MiddleName], [LastName]}, " "))`
- **Camel case to title case:** `Table.AddColumn(Source, "TitleCase", each Text.Combine(Splitter.SplitTextByCharacterTransition({"a".."z"}, {"A".."Z"})([Column]), " "))`
- **Soundex encoding:** `Table.AddColumn(Source, "Soundex", each Text.Start(Text.Upper([Name]), 1) & Text.Combine(List.Transform(Text.ToList(Text.Upper(Text.RemoveRange([Name], 0, 1))), each if _ = "A" or _ = "E" or _ = "I" or _ = "O" or _ = "U" or _ = "H" or _ = "W" or _ = "Y" then "" else if _ = "B" or _ = "F" or _ = "P" or _ = "V" then "1" else if _ = "C" or _ = "G" or _ = "J" or _ = "K" or _ = "Q" or _ = "S" or _ = "X" or _ = "Z" then "2" else if _ = "D" or _ = "T" then "3" else if _ = "L" then "4" else if _ = "M" or _ = "N" then "5" else if _ = "R" then "6" else ""), ""))`

23. Advanced List Operations

- **List cartesian product:** `List.Generate(() => [i=0, j=0, res=null], each [i] < List.Count(List1), each if [j] < List.Count(List2) - 1 then [i=[i], j=[j]+1, res={List1{[i]}, List2{[j]+1}}] else [i=[i]+1, j=0, res={List1{[i]+1}, List2{0}}]], each [res])`
- **List running total:** `List.Generate(() => [i=0, total=0], each [i] < List.Count(Source), each [i=[i]+1, total=[total] + Source{[i]}], each [total])`
- **List cumulative product:** `List.Generate(() => [i=0, product=1], each [i] < List.Count(Source), each [i=[i]+1, product=[product] * Source{[i]}], each [product])`
- **List moving average:** `List.Generate(() => [i=0, avg=null], each [i] < List.Count(Source), each [i=[i]+1, avg=List.Average(List.Range(Source, Number.Max(0, [i]-2), Number.Min(3, List.Count(Source)-[i]+2)))]], each [avg])`

24. Advanced Table Manipulations

- **Pivot with multiple value columns:** `Table.Pivot(Source, List.Distinct(Source[Category]), {"Category"}, {"Sales", "Profit"}, each {List.Sum([Sales]), List.Sum([Profit])})`
- **Dynamic query folding:** `Table.SelectRows(Source, each [Date] >= Date.From(DateTime.LocalNow()) - #duration(30, 0, 0, 0))`
- **Table normalization:** `Table.NormalizeNestedRecords(Source, {"NestedColumn"}, List.Transform(Record.FieldNames(Source{0}[NestedColumn]), each {_, "NestedColumn." & _}))`
- **Table denormalization:** `Table.Group(Source, {"ID"}, {"Attributes", each _, type table [ID=number, Attribute=text, Value=text]})`
- **Incremental refresh logic:** `Table.SelectRows(Source, each [Date] > #date(2022, 1, 1) and [Date] <= Date.From(DateTime.LocalNow()))`

25. Advanced Error Handling and Debugging

- **Custom error messages:** `try [RiskyOperation] otherwise error [Column] & "operation failed: " & _)`
- **Catch and log errors:** `Table.AddColumn(Source, "ErrorLog", each try [RiskyOperation] otherwise "Error: " & _)`
- **Conditional error handling:** `if [Condition] then try [Operation1] otherwise [Fallback1] else try [Operation2] otherwise [Fallback2]`
- **Debug.Assert:** `let assert = (condition as logical, message as text) => if condition then true else error message, Source = assert([Column] > 0, "Column must be positive") in Source`

26. Advanced Type System and Metadata

- **Custom type definition:** `type [ID = Int64.Type, Name = Text.Type, Sales = Currency.Type]`
- **Type compliance check:** `Table.TransformColumns(Source, {"Column", each if _ is number then _ else null})`
- **Metadata annotation:** `Value.ReplaceMetadata(Source, [SourceSystem = "CRM", LastRefresh = DateTime.LocalNow()])`
- **Extract table metadata:** `Table.Schema(Source)`

27. Advanced Function Techniques

- **Function memoization:** `let memoize = (f as function) => let cache = [] in (x) => if x in Record.FieldNames(cache) then Record.Field(cache, x) else let value =`

- `f(x) in (cache := Record.AddField(cache, x, value)) {x}, factorial_memo = memoize((n) => if n <= 1 then 1 else n * factorial_memo(n-1)) in factorial_memo`
- **Function composition:** `let compose = (f, g) => (x) => f(g(x)), double = each _ * 2, square = each _ * _, doubleSquare = compose(double, square) in doubleSquare`
- **Partial function application:** `let partial = (f as function, x) => (y) => f(x, y), add = (x, y) => x + y, add5 = partial(add, 5) in add5`

28. Advanced Data Source Techniques

- **Parameterized data source:** `let Source = (server as text, database as text) => Sql.Database(server, database) in Source`
- **Dynamically generated SQL query:** `let query = "SELECT * FROM " & TableName & " WHERE Date >= '" & Date.ToText(StartDate, "yyyy-MM-dd") & "'" in Value.NativeQuery(SqlServer.Database(server, database), query)`
- **Multi-source data blend:** `Table.Join(WebSource, "ID", SqlSource, "ID", JoinKind.FullOuter)`

29. Performance Optimization Techniques

- **Table.Buffer for query folding:** `let buffered = Table.Buffer(LargeTable) in Table.SelectRows(buffered, each [Column] > 100)`
- **Disable background refresh:** `let Source = Sql.Database(server, database, [Query="SELECT TOP 100 * FROM Table"]), DisableBackgroundRefresh = Value.Metadata(Value.ReplaceMetadata(Source, [IsParameterQuery=true])) in DisableBackgroundRefresh`
- **Optimize data types:** `Table.TransformColumnTypes(Source, List.Transform(Table.ColumnNames(Source), each {_, if Text.StartsWith(_, "Date") then Date.Type else if Text.StartsWith(_, "Is") then Logical.Type else Any.Type}))`

30. Advanced Analytics

- **Percentile calculation:** `let sorted = List.Sort(ColumnAsList), index = Number.RoundDown(0.9 * List.Count(sorted)) in sorted{index}`
- **Linear regression:** `let x = Source[X], y = Source[Y], n = List.Count(x), sumX = List.Sum(x), sumY = List.Sum(y), sumXY = List.Sum(List.Transform({0..n-1}, (i) => x{i} * y{i})), sumX2 = List.Sum(List.Transform(x, each _ * _)), slope = (n*sumXY - sumX*sumY) / (n*sumX2 - sumX*sumX), intercept = (sumY - slope*sumX) / n in [Slope=slope, Intercept=intercept]`
- **K-means clustering:** `let centroids = List.Random(Source[Column], k), assign = (point) => List.MinN(List.Transform(centroids, (c) => [Centroid=c, Distance=Number.Abs(point - c)]), 1){0}[Centroid] in Table.Group(Table.AddColumn(Source, "Cluster", each assign([Column])), "Cluster", {"Points", each [Column], type list}, {"NewCentroid", each List.Average([Points]), type number}))`

